Theoretical Example

```
import math as m
# ---------------- #

def normalshockrelations(M,gamma,cp,R):
      '''
      Return : m1,m2,rhoratio,pratio,Tratio,p0ratio
      '''

      m1 = M

      #Mach Relation M2
      m2 = m.sqrt( (1+0.5*(gamma-1)*M**2)/(gamma*M**2 - 0.5*(gamma-1)))

      #Density Relation
      rhoratio = ((gamma+1)*M**2)/(2+(gamma-1)*M**2)

      #Pressure Relation
      pratio = 1 + ((2*gamma)*(gamma+1)**-1)*(M**2 -1)

      #Temperature Relation
      Tratio = (1-gamma+2*gamma*M**2)*(2+(gamma-1)*M**2)/(
((gamma+1)**2)*M**2 )

      #Entropy Change
      dels = cp*m.log(Tratio) - R*m.log(pratio)

      #stagnation pressure ratio
      p0ratio = m.exp(-(dels)/R)

      # #P01/P1
      # p01p1ratio = (1+ 0.5*(gamma-1)*M**2)**(gamma/gamma-1)

      # #P01/P2
      # p02p2ratio = (1+ 0.5*(gamma-1)*m2**2)**(gamma/gamma-1)

      return m1,m2,rhoratio,pratio,Tratio,p0ratio


def isentropic_pressure(p,M,gamma):
```

```
        return p*(1 + 0.5*(gamma-1)*M**2)**(gamma/(gamma-1))


M1 = 3
p1 = 1 # Pa (N/m^2)
gamma = 1.4
R = 287.05 # J/(kgK)
cp = gamma*R/(gamma-1)

result = normalshockrelations(M1,gamma,cp,R)

p01 = isentropic_pressure(p1,M1,gamma)

print(f"p_01 = {p01} Pa")

M2 = result[1]
p2 = result[3]*p1

# p02 = p01*result[5]

p02 = p2*(1 + 0.5*(gamma-1)*M2**2)**(gamma/(gamma-1))

print(f"M2 = {M2}")

print(f"p2 = {p2} Pa")

print(f"p_02 = {p02} Pa")
```

Converging the Solution

```
import os
import glob
import numpy as np
import matplotlib.pyplot as plt

def plot_coord_vs_pressure_across_res(results_all, selected_prefix,
coord='x'):
      """
      Plots a chosen coordinate (x or y) vs pressure for a specified
prefix,
      gathering data from different resolution runs (e.g., Nx1, Nx2, Nx4,
...).

      Parameters:
      results_all (dict): Dictionary with keys like "results_Nx1",
"results_Nx2", etc.
                          Each value is a sub-dictionary where keys are
prefixes and
                          values are numpy arrays with columns [x, y, p, T,
u_x, u_y].
      selected_prefix (str): The prefix key of interest (e.g.,
"entryWallBottom").
      coord (str): Coordinate to plot against pressure. Should be 'x' or
'y' (default 'x').
      """
      # Determine which column index to use for the coordinate: 0 for x, 1
for y.
      coord = coord.lower()
      if coord not in ['x', 'y']:
      raise ValueError("The coordinate must be either 'x' or 'y'")

      coord_index = 0 if coord == 'x' else 1

      plt.figure(figsize=(10, 6))

      # Iterate through each resolution result
      for res_key, res_data in results_all.items():
      if selected_prefix in res_data:
            arr = res_data[selected_prefix]
            # Extract coordinate values and pressure values:
```

```
            coord_vals = arr[:, coord_index]   # x if coord_index=0 else y
            pressure = arr[:, 2]                # pressure is the 3rd column
(index 2)

            # Label based on resolution (e.g., "results_Nx1")
            plt.plot(coord_vals, pressure, marker=',', linestyle='-',
label=res_key)
      else:
            print(f"Prefix '{selected_prefix}' not found in {res_key}")

      plt.xlabel(f"{coord.upper()} [m]")
      plt.ylabel('Pressure [Pa]')
      plt.plot(0.6, 12.06, 'rx', label='Theoretical Stagnation Pressure,
P_0', markersize=8)  # Example experimental data point
      plt.title(f"Spatial Distribution of Pressure Along {coord.upper()},
Surface: {selected_prefix} (All Resolutions)")
      plt.legend()
      plt.grid(True)
      plt.tight_layout()
      plt.show()

def gather_data(folder_path):
      """
      Reads pairs of files named <prefix>_p_T.xy and <prefix>_U.xy
      from a given folder. Extracts columns:
      x, y, p, T, u_x, u_y
      and returns a dictionary keyed by the prefix, with values
      being a numpy array of shape (N, 6).
      """
      # Find all files that end with '_p_T.xy'
      pT_files = glob.glob(os.path.join(folder_path, '*_p_T.xy'))

      # Dictionary to hold combined data for each prefix
      data_dict = {}

      # Loop over each p_T file
      for pT_file in pT_files:
      # Derive a matching U-file name by replacing '_p_T.xy' with '_U.xy'
      prefix = pT_file.replace('_p_T.xy', '')
      u_file = prefix + '_U.xy'

      # Check if the matching U-file exists
      if os.path.isfile(u_file):
```

```
            # Load the p_T file
            # Columns: x, y, z, p, T  -> indices: 0,1,2,3,4
            try:
                  pT_data = np.loadtxt(pT_file)
            except Exception as err:
                  print(f"Error reading {pT_file}: {err}")
                  continue

            # Load the U file
            # Columns: x, y, z, u_x, u_y, u_z -> indices: 0,1,2,3,4,5
            try:
                  u_data = np.loadtxt(u_file)
            except Exception as err:
                  print(f"Error reading {u_file}: {err}")
                  continue

            # Check if both files have same number of rows
            if pT_data.shape[0] != u_data.shape[0]:
                  print(f"Warning: row mismatch between {pT_file} and
{u_file}")
                  # You could decide to skip or handle partial matches
differently
                  continue

            # Extract the columns you care about:
            #   x = pT_data[:, 0]
            #   y = pT_data[:, 1]
            #   p = pT_data[:, 3]
            #   T = pT_data[:, 4]
            #   u_x = u_data[:, 3]
            #   u_y = u_data[:, 4]
            # (We are ignoring z and u_z based on your requirement)

            combined_data = np.column_stack((
                  pT_data[:, 0],   # x
                  pT_data[:, 1],   # y
                  pT_data[:, 3],   # p
                  pT_data[:, 4],   # T
                  u_data[:, 3],     # u_x
                  u_data[:, 4]      # u_y
            ))

            # Store in dictionary keyed by prefix (basename without
```

```
suffixes)
            # For example, if pT_file is 'entryWallBottom_p_T.xy',
            # prefix = 'entryWallBottom'
            # data_dict['entryWallBottom'] = combined_data
            base_name = os.path.basename(prefix)
            data_dict[base_name] = combined_data

      else:
            print(f"No matching U file found for {pT_file}")

      return data_dict

def plot_pressure_vs_coordinates(data_dict, prefixes=None):
      """
      Plots x vs pressure and y vs pressure for specified prefixes.

      Parameters:
      data_dict (dict): Dictionary with keys as prefix strings and values
as numpy arrays
                        with columns [x, y, p, T, u_x, u_y].
      prefixes (list, optional): List of prefix keys to plot. If None, all
keys are plotted.
      """
      if prefixes is None:
      prefixes = list(data_dict.keys())

      # Plot x vs pressure
      plt.figure(figsize=(10, 6))
      for prefix in prefixes:
      data = data_dict[prefix]
      x = data[:, 0]   # x-values
      p = data[:, 2]   # pressure (3rd column)
      plt.plot(x, p, marker=',', linestyle='-', label=prefix)
      plt.xlabel('x [m]')
      plt.ylabel('Pressure [Pa]')
      plt.title('Spatial Distribution of Pressure on Horizontal Surfaces')
      plt.plot(0.6, 12.06, 'rx', label='Theoretical Stagnation Pressure,
P_0', markersize=8)  # Example experimental data point
      plt.legend()
      plt.grid(True)

      # Plot y vs pressure
      plt.figure(figsize=(10, 6))
```

```
      for prefix in prefixes:
      data = data_dict[prefix]
      y = data[:, 1]   # y-values
      p = data[:, 2]   # pressure (3rd column)
      plt.plot(y, p, marker=',', linestyle='-', label=prefix)
      plt.xlabel('y [m]')
      plt.ylabel('Pressure [Pa]')
      plt.title('Spatial Distribution of Pressure on Vertical Surfaces')
      plt.plot(0.6, 12.06, 'rx', label='Theoretical Stagnation Pressure,
P_0', markersize=8)  # Example experimental data point
      plt.legend()
      plt.grid(True)

      plt.show()

def truncate_stepFaceVert(results_all, y_threshold=0.2):
      """
      For each resolution (key) in results_all, if the prefix
"stepFaceVert" exists,
      truncate its data so that only rows with y <= y_threshold are kept.

      Parameters:
      results_all (dict): Dictionary with keys (like "results_Nx1") mapping
to sub-dictionaries
                        where each sub-dictionary has keys for each prefix
and values as numpy arrays
                        with columns [x, y, p, T, u_x, u_y].
      y_threshold (float): The y-value threshold above which the data is
truncated.
                        (Default is 0.2)
      """
      for res_key, data_dict in results_all.items():
      if "stepFaceVert" in data_dict:
            data = data_dict["stepFaceVert"]
            # Option 1: If the data is sorted by y in increasing order:
            idx = np.searchsorted(data[:, 1], y_threshold, side='right')
            truncated_data = data[:idx, :]

            # Option 2: (Alternate) if you want to filter every row with y
> threshold,
            # uncomment the next two lines and comment the searchsorted
lines above.
            # truncated_data = data[data[:, 1] <= y_threshold, :]
```

```
            # idx = truncated_data.shape[0]

            data_dict["stepFaceVert"] = truncated_data
            print(f"{res_key}: 'stepFaceVert' truncated from
{data.shape[0]} to {truncated_data.shape[0]} rows (y <= {y_threshold}).")
      else:
            print(f"{res_key}: 'stepFaceVert' not found.")

def plot_truncated_stepFaceVert_single_res(results_all, resolution_key,
coord='y'):
      """
      Plots the truncated 'stepFaceVert' data for a given resolution.

      Parameters:
      results_all (dict): Dictionary with keys like "results_Nx1",
"results_Nx2", etc.
      resolution_key (str): The key for the resolution to plot (e.g.,
"results_Nx1").
      coord (str): Which coordinate to plot against Pressure ('x' or 'y').
Default is 'y'.
      """
      if resolution_key not in results_all:
      print(f"Resolution key {resolution_key} not found.")
      return

      data_dict = results_all[resolution_key]
      if "stepFaceVert" not in data_dict:
      print("stepFaceVert data not found for", resolution_key)
      return

      # Get the truncated data
      data = data_dict["stepFaceVert"]
      if data.size == 0:
      print("Truncated data for stepFaceVert is empty for", resolution_key)
      return

      # Determine which column to use: 0 for x, 1 for y.
      coord = coord.lower()
      if coord not in ['x', 'y']:
      raise ValueError("Coordinate must be 'x' or 'y'")

      coord_index = 0 if coord == 'x' else 1
```

```
        # Extract coordinate and pressure (pressure is always column 2)
        coord_vals = data[:, coord_index]
        pressure = data[:, 2]

        plt.figure(figsize=(8, 5))
        plt.plot(coord_vals, pressure, marker=',', linestyle='-',
label='stepFaceVert')
        plt.xlabel(f"{coord.upper()} [m]")
        plt.ylabel("Pressure [Pa]")
        plt.title('Spatial Distribution of Pressure on Vertical Step Face')
        plt.plot(0.0, 12.06, 'rx', label='Theoretical Stagnation Pressure,
P_0', markersize=8)  # Example experimental data point
        plt.legend()
        plt.grid(True)
        plt.tight_layout()
        plt.show()



Res = [1, 2, 4, 8]

results_all = {}

for r in range(len(Res)):
        # Example usage:
        folder_path = rf"C:\Users\TerryM\Documents\University of Texas\Intro
to
CFD\OF-3\mach-3-wind-tunnel-with-step-Nx{Res[r]}\postProcessing\singleGraph
\4"  # Update to your directory
        results = gather_data(folder_path)
        print("results keys:", list(results.keys()))
        # Save results with key e.g., 'results_Nx1'
        results_all[f"results_Nx{Res[r]}"] = results

        # 'results' is a dictionary keyed by prefix, e.g.:
        # {
        #   "entryWallBottom": array([[x1, y1, p1, T1, u_x1, u_y1],
        #                             [x2, y2, p2, T2, u_x2, u_y2],
        #                             ... ]),
        #   "entryWallTop":     array(...),
        #   ...
        # }
```

```
# Assuming results_all is your dictionary containing the data for different
resolutions:
truncate_stepFaceVert(results_all, y_threshold=0.2)

# Example usage for a specific resolution (say, Nx1):
plot_truncated_stepFaceVert_single_res(results_all, "results_Nx8",
coord='y')

results = gather_data(folder_path)
print("results keys:", list(results_all.keys()))

# You can now access the data for each prefix:
for res_key, data_dict in results_all.items():
      print(f"\n{res_key}:")
      for prefix, arr in data_dict.items():
      print(f"Prefix: {prefix}, Data shape: {arr.shape}")
      # arr has columns: [x, y, p, T, u_x, u_y]
      # e.g. show the first row
      print("First row:", arr[0, :])

# Example usage:
if __name__ == '__main__':
      # Assume 'data_dict' holds your assembled data from the previous
step.
      # For example, data_dict might be structured as:
      # {
      #     'entryWallBottom': np.array([...]),
      #     'entryWallTop': np.array([...]),
      #     'stepFaceHoriz': np.array([...]),
      #     ...
      # }

      # If you want to plot all prefixes, just pass data_dict.
      # To only plot selected prefixes, update the list e.g.,
['entryWallBottom', 'entryWallTop']
      selected_prefixes = ['stepFaceVert']

      # Call the function with your dictionary and the selected prefixes.
      plot_pressure_vs_coordinates(results, selected_prefixes)

# Example usage:
if __name__ == '__main__':
```

```
      # Assuming you have built a results_all dictionary from your multiple
resolutions.
      # For example:
      # results_all = {
      #     "results_Nx1": { "entryWallBottom": np.array([...]),
"entryWallTop": np.array([...]), ... },
      #     "results_Nx2": { "entryWallBottom": np.array([...]),
"entryWallTop": np.array([...]), ... },
      #     "results_Nx4": { ... },
      #     "results_Nx8": { ... },
      # }

      # Specify the prefix you want to plot.
      selected_prefix = "entryWallBottom"

      # Plot x vs pressure for the given prefix
      plot_coord_vs_pressure_across_res(results_all, selected_prefix,
coord='x')

      selected_prefix = "entryWallTop"
      plot_coord_vs_pressure_across_res(results_all, selected_prefix,
coord='x')

      selected_prefix = "stepFaceHoriz"
      plot_coord_vs_pressure_across_res(results_all, selected_prefix,
coord='x')

      selected_prefix = "stepFaceVert"
      plot_coord_vs_pressure_across_res(results_all, selected_prefix,
coord='y')

      # To plot y vs pressure instead, call:
      # plot_coord_vs_pressure_across_res(results_all, selected_prefix,
coord='y')
```

Parallel Performance Plot

```
import numpy as np
import matplotlib.pyplot as plt

processors = np.array([1, 2, 4, 8, 16, 32, 56])  # P
actual_speedup = np.array([1.0, 2.150538, 4.223963, 8.039254, 13.36682,
21.37129, 26.80988])

x = np.linspace(1, 60, 200)
ideal_speedup = x             # Ideal = y = P
speedup_80   = 0.8 * x        # 80% efficient
speedup_60   = 0.6 * x        # 60% efficient

plt.figure(figsize=(8, 6))

# Plot actual
plt.plot(processors, actual_speedup, 'rs-', label='actual (1 M cells)')
# Plot the theoretical lines
plt.plot(x, ideal_speedup, 'k-',  label='ideal')
plt.plot(x, speedup_80,       'k--', label='80% efficient')
plt.plot(x, speedup_60,       'k-.', label='60% efficient')

plt.xlim(0, 60)
plt.ylim(0, 70)
plt.xlabel('# of processors, P')
plt.ylabel('Speed-up, T(1)/T(P)')
plt.title('Speed-up vs. Number of Processors')
plt.grid(True)
plt.legend(loc='upper left')
plt.show()
```